Document Intelligence System

Product requirements and phased implementation plan

Version 1.0 | Prepared 8 October 2026 | Release deadline 15 November 2026

The product we will build

Build a web application that lets a user upload documents, ask questions about them, and receive answers supported by clickable evidence. When the uploaded documents do not support an answer, the application must say so. The product should make it easier to find and verify information than manually searching a folder of PDFs.

The release should be a deployed, portfolio-quality MVP with a reproducible evaluation report. Prioritize reliable retrieval, correct citations, safe access, and a working demo. Add autonomous tools and broad multimodal support after the deadline.

Who it serves

- Students finding definitions and policies in course documents.
- Researchers looking for a passage across a small collection of reports.
- Small teams checking information in approved manuals and internal documents.

Planning assumptions

Plan for one developer with basic Python knowledge, approximately 15–20 hours per week, and a small demonstration budget. The schedule allocates about 95 hours including a 9-hour contingency. There are 38 elapsed days from 8 October to 15 November, or 39 calendar dates including both endpoints. A developer learning Python and web development from scratch will need more time or a smaller scope.

The initial corpus is English, non-sensitive, and modest: up to 50 documents or 1,000 total pages per workspace, whichever limit is reached first. The November release is a limited beta rather than an enterprise service. These are product decisions for planning, not performance results.

What makes this project credible

Its strongest differentiator is evidence: a repeatable benchmark showing how retrieval changes affect answer quality, a source viewer that makes claims verifiable, and documented failures. Do not claim an improvement such as 71% to 86% until measured on the same held-out dataset.

1 Product scope and requirements

A successful session is: sign in, upload a document, see it become ready, select the collection, ask a question, and inspect the original source. Keep this flow short and predictable.

ID	Priority	Requirement and acceptance
FR01	Must	Sign in and isolate user-owned workspaces. A second user cannot list, retrieve, or view the first user's files.
FR02	Must	Upload English text PDFs, TXT, and Markdown. Limit each file to 10 MB and 100 PDF pages; reject unsupported or encrypted files clearly.
FR03	Must	Show queued, processing, ready, and failed states. Retry a failed job without duplicating chunks.
FR04	Must	Ask questions across ready documents in a selected workspace, with an optional document filter.
FR05	Must	Return an answer with citations to document, PDF page or text heading, and a supporting excerpt.
FR06	Must	State when evidence is missing or contradictory. Never present an invented citation.
FR07	Must	Delete a document and prevent subsequent search or source access immediately.
FR08	Must	Run a frozen evaluation set and save per-question evidence, configuration, quality, latency, and cost.
FR09	Should	Collect helpful or unhelpful feedback and offer simple session history.
FR10	Optional	Add OCR for English scans and reranking only after measured benefit and resource checks.

Scope excluded from the November release

No web crawling, agent tool execution, email or cloud-drive connectors, voice interface, image reasoning, handwritten document support, enterprise sharing roles, fine-tuning, or arbitrary spreadsheet calculations. DOCX ingestion and broad multilingual retrieval belong to the next release unless the core system is finished early.

User stories

- As a student, I can ask "What is the attendance requirement?" and open the exact policy passage.
- As a researcher, I can filter to one report and verify the cited page.
- As a user, I receive "I could not find this in your selected documents" for an unsupported question.
- As the builder, I can compare retrieval configurations without changing the test questions.

2 Gaps in the original idea and corrections

The reference captures the central RAG loop and rightly emphasizes evaluation. The following additions turn that outline into a buildable product. The supplied image does not specify an existing technology stack, so the choices later in this document are proposed defaults rather than confirmed replacements.

Gap	Consequence	Correction
RAG described as agentic AI	May encourage unnecessary autonomous agents.	Use a deterministic search-and-answer workflow. RAG supplies evidence; an agent independently chooses actions.
No document parsing policy	Scans, tables, and encrypted PDFs can silently fail.	Validate formats, preserve page provenance, measure extraction quality, and report unsupported files.
Embeddings treated as complete search	Exact names, codes, and uncommon terms can be missed.	Benchmark semantic search against hybrid keyword plus vector search.
Citations without validation	A believable page link may not support the claim.	Resolve citations from retrieved IDs and manually evaluate claim support.
Hit rate as the only metric	One relevant chunk can conceal missing evidence.	Measure evidence recall, answer correctness, citation support, refusal quality, latency, and cost.
No unavailable answer behavior	The model may fill gaps from memory.	Calibrate abstention on development questions and test unsupported questions separately.
No permissions or privacy	A private passage could leak in search or logs.	Enforce owner scope before both retrieval branches and source access.
No ingestion lifecycle	Restarting a process can lose uploads or duplicate data.	Use durable jobs, idempotent processing, bounded retries, and index versions.
No delivery boundaries	New tools can consume the entire deadline.	Freeze the MVP, establish dated gates, and reserve 13–15 November for fixes.

What the system can honestly promise

It can answer from extracted evidence and show where that evidence came from. Retrieval, parsing, and models can all fail. It cannot guarantee truth, perfectly understand every PDF table, or replace expert review. Display limitations in the product when relevant and report benchmark failures in the project documentation.

3 Recommended modern technology stack

Choose components that expose the engineering decisions and minimize separate services. This is a recommendation for this project and deadline, not a claim that one stack is universally best. Official documentation was checked on 8 October 2026; pin tested package and model versions during setup. Sources S1–S9 appear at the end.

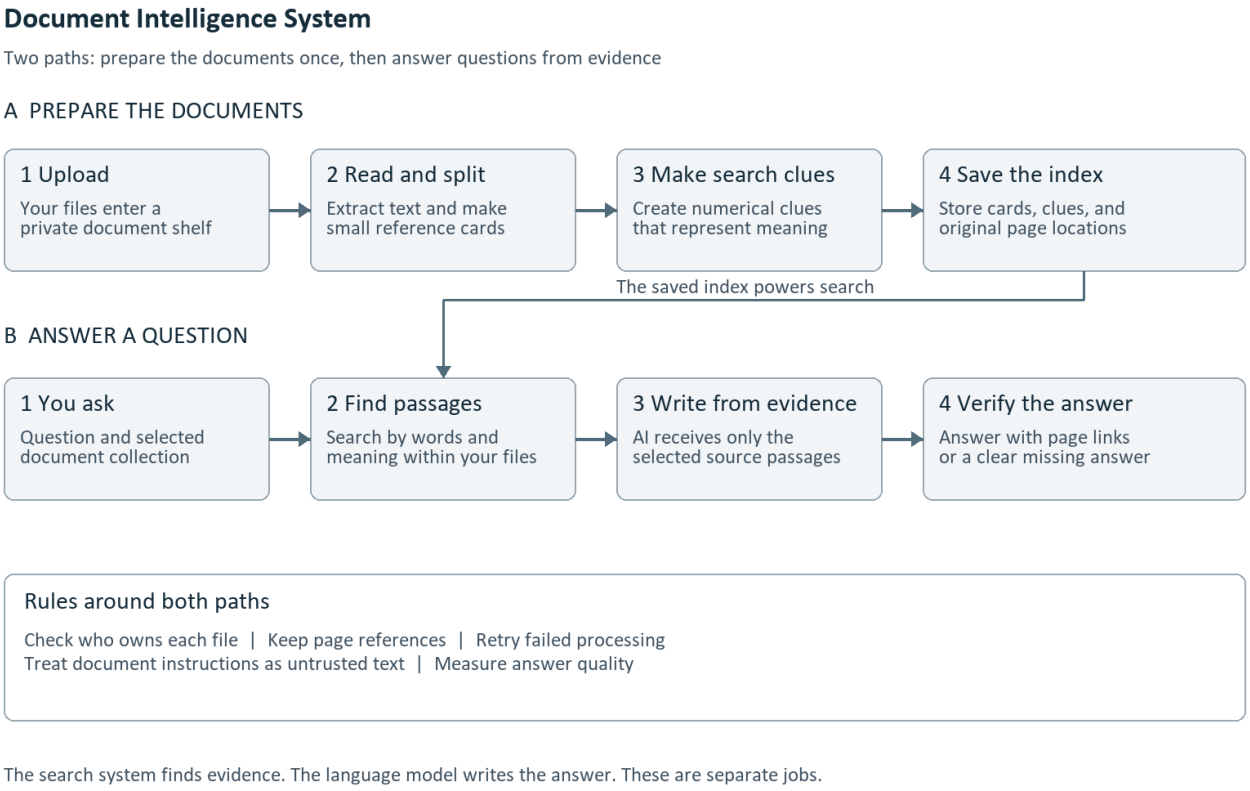
Layer	Recommended choice	Reason and alternative
Web interface	React, TypeScript, Vite	Modern browser app with clear types; avoid a second backend layer. Use Next.js later if server rendering is needed. [S1]
API	FastAPI and Pydantic	Python API with validated inputs and outputs. Keep retrieval modules independent of routes. [S2]
Data and search	Supabase PostgreSQL with pgvector and full-text search	One database for users, metadata, keyword ranking, and embeddings. Dedicated Qdrant is a later scale option. [S3,S4]
Login and files	Supabase Auth and private Storage	Share workspace permissions across data and files; proxy source access through the API to enforce deletion. [S5]
Text extraction	pypdf for text PDFs	Start with page-aware extraction. Docling is the optional upgrade for OCR and layout or table parsing; budget extra memory and setup. [S6,S12]
Generation	Gemini 3.8 Flash via google-genai	Current stable candidate listed in official docs. Benchmark availability, cost, citations, and latency before locking it; keep a provider adapter. [S7]
Embeddings	gemini-embedding-001 for text	Available text baseline. Evaluate gemini-embedding-2 only if it improves the workload; do not add multimodal scope merely because it is newer. [S8]
Ingestion jobs	Postgres jobs table and one Python worker	Durable work without an extra broker. Use leases and retries; move to a queue system if operations require it.
Evaluation	pytest, JSONL gold set, optional Ragas	Manual truth and citation labels are primary. Ragas adds automated faithfulness diagnostics. [S9]
Delivery	Docker, GitHub Actions, static UI plus API and worker host	Reproducible runtime and checks. Use uv with a lockfile for Python and the frontend lockfile for JS.

What to upgrade in a typical prototype

If you were considering local FAISS or Chroma, prefer PostgreSQL here for permissions and metadata in one place. If you were considering a notebook-only interface, move the user flow into a web app. If you were considering LangChain, LlamaIndex, or LangGraph, add a framework only for a specific missing capability. None is mandatory for this MVP. Keep a small explicit pipeline that is easy to test.

4 Architecture explained for a beginner

Imagine a helpful librarian. Uploading a document puts a book on the shelf. The system creates small reference cards from its pages, records where every card came from, and organizes them so the librarian can find relevant passages quickly. The AI then writes an answer using only those passages.



The important words

- Chunk: a small passage from a document, like a reference card.
- Embedding: numerical clues for matching meanings; it is not a replacement for the original text.
- Vector search: finding passages with similar meaning. Keyword search: finding matching words.
- RAG: find evidence first, then give it to the AI to help produce a supported answer.
- Citation: a link back to the evidence. The database preserves it even when the AI makes a mistake.

Example user journey

You upload College Handbook.pdf and ask “How many days can I miss?” Search finds the attendance paragraph even if the exact words differ. The model summarizes the rule and returns a citation such as “College Handbook.pdf, page 12.” That page number is illustrative; the system must use the real stored source location. A question about a rule absent from the file produces a missing-evidence response.

5 How the processing and answer pipelines work

Document preparation

1. Authenticate the upload; check extension, MIME type, file signature, size, and workspace quota. Store the file in a private bucket with a generated ID.

2. Create a durable job. A worker claims it with a lease, updates processing state, and extracts text page by page. Flag scans or badly extracted content rather than indexing blank pages.

3. Preserve document version, PDF page index, headings where available, and paragraph offsets. The first PDF page is page 1; printed page labels may differ.

4. Begin with about 500-token chunks and 75-token overlap. Avoid splitting short paragraphs; split across pages only when the page mapping remains accurate.

5. Generate one embedding per chunk, using the selected model and matching query/document retrieval modes. Store dimension, model identifier, parser version, and chunk configuration.

6. Write chunks into a new index version. Publish it as active only after successful completion. Failed runs remain invisible to search; retry up to three times with backoff.

Answering a question

1. Validate session and workspace ownership. Exclude failed, deleted, and inactive document versions before retrieval.

2. For a follow-up, rewrite the question using a small bounded conversation history. History helps interpret the question; it is not evidence.

3. Run keyword and vector searches with the same owner and document filters. Start with 20 candidates per branch, merge by reciprocal rank fusion, and retain up to 20 unique candidates.

4. Optionally rerank the candidates if the benchmark justifies it. Select about 5 passages within a configurable context token budget.

5. Give the model the question, labeled passages, and an instruction to answer only from evidence, explain contradictions, and abstain when unsupported.

6. Request structured output containing answer, evidence status, and cited chunk IDs. Validate IDs against retrieved context; resolve document names and locations on the server.

7. Show the answer and excerpts. Record timings, token usage, model/configuration versions, and feedback without saving sensitive content by default.

Starting parameters are experiments

Chunk size, overlap, candidate counts, and refusal rules are defaults to test, not universal settings. A similarity score is not a calibrated confidence percentage. Improve refusal behavior using development questions and human labels. Do not rely on a model's self-reported confidence to decide truth.

6 Technical design and service contracts

Use one application backend and one ingestion worker from the same repository. The browser talks to the API for protected operations. Generation and embedding keys stay on the server. Private files and relational metadata live in Supabase; documents sent to a model provider cross an external service boundary.

Minimum data model

Entity	Important fields
workspaces	id, owner_user_id, name, created_at
documents	id, workspace_id, file_key, filename, SHA256, size, status, active_version, deleted_at
document_versions	id, document_id, parser_version, chunk_config, embedding_model, embedding_dimension, status
chunks	id, version_id, workspace_id, text, PDF page start/end or heading/offset, embedding, full-text index
ingestion_jobs	id, document_id, status, attempt_count, lease_expiry, progress, error_code
chat sessions and messages	session_id, workspace_id, user question, answer, cited IDs, created_at; optional persistence
evaluation cases and runs	case_id, source versions, gold evidence spans, reference answer, split, config, results
request events	request_id, timings, token counts, estimated cost, failure code; no raw text by default

Minimum API

POST /documents creates an upload and job and returns document_id, job_id, and status. GET /documents and GET /jobs/{id} show authorized files and progress. POST /chat accepts workspace_id, question, optional document_ids, and session_id. It returns answer, evidence_status, citations, and request_id. GET /sources/{chunk_id} validates access and returns an excerpt plus an authenticated source-view route. DELETE /documents/{id} tombstones the file and schedules physical cleanup. GET /health reports readiness without exposing secrets.

Correctness rules

- Derive the authenticated user from a verified token, never from a user_id sent by the browser.
- Deduplicate by workspace plus content hash and processing version. A retry must not create duplicate active chunks.
- Keep chunk IDs and source locators stable within a document version. Model citations may refer only to context IDs from the current request.
- Do not mix embeddings from different models, even if dimensions match. Rebuild the index and atomically switch the active version when changing models.
- Use exact vector search for the small initial corpus. Add an HNSW index after measurements; verify filtered recall if using approximate search. [S4]

7 Evaluation plan and release quality targets

Create 50 manually verified cases from 10–15 representative documents. Allocate 30 to development and 20 to a frozen final test set. Group related questions by topic or source passage to avoid near-duplicate leakage. This is a portfolio benchmark, not statistical proof of general reliability.

Dataset composition

Development set: 12 direct factual, 6 paraphrase or exact-code, 4 multi-passage, 2 contradictory-source, and 6 unanswerable cases. Final set: 8 direct factual, 4 paraphrase or exact-code, 2 multi-passage, 2 contradictory-source, and 4 unanswerable cases. Label each with the corpus version, reference answer or refusal, exact evidence passage and source location, and required facts. This gives 36 answerable and 14 unanswerable cases overall.

Gold evidence uses source spans rather than chunk IDs, so chunking experiments remain comparable. A retrieved chunk counts as a hit only when it contains the annotated supporting span under the documented match rule. Report contradictory-source cases separately; both conflicting passages are required evidence.

Metric	Definition	Proposed release target
Hit@5	Answerable cases with at least one gold evidence span in the top five passages.	At least 14 of 16 answerable final cases.
Evidence recall@5	Fraction of required gold evidence spans found; report multi-passage cases separately.	Measure and publish; do not hide partial evidence behind hit rate.
Answer correctness	Human rubric: 2 fully correct, 1 partially correct, 0 incorrect; required facts and conflicts matter.	At least 13 of 16 answerable final cases score 2.
Citation support	Cited factual claims genuinely supported by the linked passages; include unsupported uncited claims as failures.	At least 90% of reviewed factual claims supported.
Citation resolution	Every displayed citation maps to an authorized retrieved chunk and usable source location.	100% in automated release tests.
Abstention	No invented answer on unsupported final questions.	All 4 final unanswerable cases correctly refused.
Latency	Warm end-to-end completed-answer p95 over 100 requests at five concurrent users.	At most 10 seconds on declared hosting and corpus.
Ingestion	Elapsed time for a 20-page, text-only reference PDF on declared hardware.	At most 120 seconds; report OCR separately.

Experiments and honest reporting

Compare A fixed-size chunks plus vector retrieval, B paragraph or heading-aware chunks with the same embedding and generator, C hybrid search, and D optional reranking. Change one factor at a time; choose on the development set and run the frozen test set after selection. If it fails, publish that result; later tuning needs a new held-out set. Save counts, failures, model IDs, prompts, source hashes, parameters, latency, and cost. Ragas faithfulness is a diagnostic, not a substitute for human verification. [S9]

8 Phase wise delivery calendar

All dates are in 2026 and use India Standard Time. Each phase has a demonstrable exit condition. The critical path is file extraction, provenance, retrieval, supported answers, evaluation, secure deployment, and rehearsal.

Phase and dates	Hours	Deliverable and exit gate
0 Scope and setup 8–11 October	8	Freeze MVP; create repository, lockfiles, corpus, schema, login skeleton, and a deployed hello-world API and UI.
1 Ingestion 12–18 October	17	Upload, parse, chunk, embed, and index a text PDF. Job status and retry work; source locations are correct.
2 Complete Q and A 19–25 October	18	Ask questions and inspect citations; missing-evidence behavior and document filtering work. First end-to-end demo on 25 October.
3 Benchmark and improve 26 October–1 November	16	Label 50 cases, compare retrieval approaches on the development split, and select a configuration. Keep reranking optional.
4 Secure beta 2–8 November	17	Complete login, ownership checks, deletion, reliability tests, latency measurement, and beta deployment. Feature freeze on 8 November.
5 Release preparation 9–12 November	10	Run frozen evaluation and release checks, document measured results and failures, record demo, and tag the release candidate.
Contingency 13–15 November	9	Fix release-blocking defects, rehearse, and submit or demo by 15 November. Add no new features.

Milestones you should be able to show

- 18 October: one real document can be processed and searched.
- 25 October: a user asks a question and opens supporting evidence.
- 1 November: a saved benchmark explains why the chosen retrieval method was selected.
- 8 November: a deployed beta safely separates two users.
- 12 November: release candidate, measured report, and backup demo are ready.
- 15 November: final project is delivered with a working link, repository, documentation, and demonstration.

If the schedule slips

Cut OCR, reranking, persistent chat history, and UI polish in that order. Keep access control, citations, abstention, deletion, and evaluation. If cross-document questions remain unreliable, explicitly narrow the release to one selected document. If a secure public deployment cannot be completed, deliver a local authenticated demo and clearly label the deployment limitation.

9 Implementation checklist for each phase

Phase 0 Establish a foundation

Learn the RAG loop, environment variables, API requests, and basic database queries. Prepare 10 legal-to-use, non-sensitive PDFs including an empty file, scan, and tricky layout. Create frontend, backend, worker, migrations, and evaluation folders. Confirm model access with one generation and one embedding call. Deploy early to discover hosting constraints before they become deadline risks.

Phase 1 Build document preparation

Implement upload validation, private storage, a documents screen, job polling, page-aware extraction, chunking, and embedding writes. Add index version publication and idempotency. Exercise restart recovery, duplicate upload, unsupported format, and empty extraction. Use 500-token chunks initially and inspect several extracted pages against the originals.

Phase 2 Complete the visible user journey

Build workspace selection, question entry, answer display, and a sources panel. Add vector retrieval first, then evidence-constrained generation and server citation resolution. Add missing-evidence and contradictory-source responses. Preserve source filters for follow-up questions. Demonstrate at least five supported answers and two unsupported questions with traceable evidence.

Phase 3 Improve using evidence

Finish the 30 development and 20 final cases; freeze the final file. Implement keyword search and rank fusion. Compare chunking and hybrid retrieval under the same model settings. Evaluate optional reranking only if retrieval errors remain and hosting can handle it. Measure costs and slow steps. Keep the simpler configuration when improvements are marginal or regressions outweigh gains.

Phase 4 Make the beta safe and recoverable

Complete authenticated user paths and database/storage access policies. Use two test accounts for list, chat, retrieval, source URL, and deletion isolation. Test prompt injection, malformed model output, provider timeouts, ingestion restarts, and cleanup retries. Deploy API and worker with proper secrets and private storage. Confirm five concurrent users do not break the chosen hosting.

Phase 5 Package and deliver

Run the frozen final set once after the configuration is selected and record actual results. Create a clear README, setup instructions, architecture explanation, troubleshooting notes, evaluation report, and limitations. Capture a three-minute demo and screenshots. Tag the release candidate by 12 November. Keep a local copy of the code, public demo corpus, benchmark, and recording for a network failure on demo day.

Daily working rhythm

Finish one small user-visible slice each session. End with a saved change and a brief note: what works, what failed, and the next task. Use the dated exit gates to decide cuts. Avoid starting another model or database migration after the 8 November feature freeze unless needed to fix a blocker.

10 Security privacy and operational reliability

Document text is untrusted

A document may say “ignore previous instructions” or contain a fake request to reveal secrets. Store and quote it as content; never give document text authority over application rules. Keep retrieved passages separate from instructions, disable tool execution, validate model output, and render answers with sanitized Markdown or plain text. Include adversarial documents in the test suite. A prompt alone does not guarantee injection resistance.

Access rules

Check workspace ownership before both keyword and vector retrieval, not after global results are selected. Apply row-level security to documents, versions, chunks, jobs, and chat records, and private bucket policies to files. Use user-scoped database requests where possible. A worker’s privileged key stays server-side; privileged API operations require explicit ownership checks because privileged credentials can bypass RLS. [S5]

Deletion and retention

Tombstone a deleted document immediately, exclude it from retrieval and source viewing, invalidate related answer caches, and enqueue deletion of original files and chunks. Target live storage cleanup within 24 hours and retry failures. Proxy file viewing through an authenticated backend route so each request checks deletion and ownership. Do not expose reusable storage URLs; material already downloaded cannot be revoked. Backups follow the chosen host retention policy, which must be disclosed before accepting private data.

External model boundary

Embeddings send extracted chunk text to the embedding provider; answer generation sends the question and selected passages. Explain this before upload, use non-sensitive demo files, and check the provider’s account-specific retention and training terms before allowing private documents. A local embedding model alone does not make generation private. A fully local mode is future scope and needs hardware benchmarking.

Recovery and observability

- Jobs use transactional claims, expiring leases, bounded retries, and idempotent writes. A restarted worker can resume a job without publishing a partial index.
- Use parser time and memory limits; reject suspicious archives and unsupported encrypted PDFs. Generated object keys prevent user filenames becoming filesystem paths.
- Set request timeouts, rate limits, per-user quotas, maximum context tokens, and a spending cap. Do not retry expensive generation indefinitely.
- Log request IDs, error codes, durations, token usage, and configuration versions. Avoid logging full files, questions, API keys, or retrieved private text.
- Keep database migrations in version control, verify backup restore on a small corpus, and keep the last working release available for rollback.

11 User experience and acceptance scenarios

Four simple screens

Sign in: a clear entry point and session handling. Document library: upload, limits, privacy notice, status, retry, and delete. Ask: selected workspace and document filter, question input, loading state, answer, and evidence. Sources: file title, excerpt, real page location, and original document access. Keep evaluation reports in the repository; an admin dashboard is optional.

Product language

- Ready: “Your document is ready for questions.”
- Scan unsupported: “This file appears to contain scanned pages. Text extraction is not available in this release.”
- Missing answer: “I could not find enough information in your selected documents to answer this.”
- Conflicting evidence: “These documents give different answers.” Show both sources.
- Provider unavailable: “The answer service is temporarily unavailable. Your document is still saved.”
- Deleted source in old history: “This source is no longer available.” Do not reuse it to answer a new question.

Acceptance scenarios

Scenario	Expected result
Valid PDF upload	Becomes ready; extracted text matches sampled pages; citations open the correct page.
Unknown or blank file	Readable rejection or failure; no blank chunks or falsely ready status.
Exact policy code and paraphrase	Both can retrieve gold evidence under the selected hybrid configuration.
Question outside the corpus	Explicit missing answer rather than an answer from general model memory.
Conflicting documents	Both passages cited and the disagreement explained.
Malicious document instruction	No tool action, secret disclosure, or bypass of evidence and ownership checks.
Second user queries first user’s IDs	Access denied for listing, chat, retrieval, jobs, source viewing, and deletion.
Worker restart or model timeout	Recoverable state and clear message; no duplicate active chunks or partial index.
Delete during or after processing	Search access stops immediately; job cannot republish the deleted file; cleanup retried.
Keyboard-only navigation	Upload controls, question input, sources, and error states are usable without a mouse.

Performance test conditions

Declare host memory and CPU, region, model IDs, corpus size, token budget, concurrency, warm versus cold status, and request count. Use 100 representative requests for the latency target and report p50 and p95. A small measured sample must not be advertised as unlimited scale. OCR processing has a separate baseline and is excluded from the text-PDF target.

12 Budget risks and decisions

Planning budget

Reserve INR 3,000–8,000 through the release for hosting, model calls, embeddings, and evaluation; this is a planning allowance, not a vendor quote. Try local development and available free tiers first, but do not depend on free quotas or sleeping services for the final demo. Check current vendor prices and account limits during setup. Supabase publishes its current plan pricing in S10.

Estimate generation cost as input tokens times input price per million plus output tokens times output price per million. Add embedding tokens, judge calls, reranking, storage, and hosting separately. For illustration only: 1,000 questions at 4,000 input tokens and 500 output tokens use 4 million input and 0.5 million output tokens before retries or evaluation. Substitute real provider prices; do not treat this as an invoice estimate.

Risk register

Risk	Early warning	Response
New developer overload	No end-to-end answer by 25 October.	Drop optional features; focus on one document type and one retrieval pipeline.
Bad PDF extraction	Missing text, wrong reading order, lost tables.	Reject poor extraction or use Docling on a tested subset; show limitations.
Retrieval misses	Good passages absent despite sensible answers.	Inspect gold evidence, chunk boundaries, exact terms, and hybrid ranking before switching generators.
Invented or weak citations	Links exist but do not support a claim.	Validate IDs, strengthen evidence prompts, and review claim support manually.
Provider changes or quota	Errors, retired model, changed behavior.	Keep an adapter, pin tested model configuration, revalidate fallback, and retain recorded demo.
Worker too expensive or slow	High memory, poor cold starts, OCR stalls.	Keep OCR optional, profile extraction, and cap file size and processing concurrency.
Access leakage	Other user's IDs return results or files.	Treat as a release blocker; test every access path with two accounts.
Benchmark tuning leakage	Final questions influence parameters.	Select only on development cases; record failed final results or create a new held-out set.

Decisions to lock during setup

Confirm weekly hours, public versus local demonstration, actual hosting allowance, and corpus availability. Test generation and embedding API access. Select one model, vector dimension, region, parser, and corpus version. If hours are below 12 per week, reduce to single-document text-PDF Q&A and 30 evaluation cases with a separate test split; preserve security and evidence behavior.

After 15 November

Extend to DOCX and controlled OCR, then better table extraction, multilingual tests, and shared-workspace roles. Consider agentic workflows only for a defined need such as suggesting document searches across approved connectors. Tool actions need permission checks and their own evaluations. Adopt a dedicated vector store only when measured scale or operational requirements justify it.

13 Definition of done and supporting references

Release checklist for 15 November 2026

- Core upload, ready-state, question, citation, missing-answer, and delete flows work.
- Two-user isolation, injection handling, restart recovery, and failure messages pass release checks.
- Measured evaluation report includes counts, configurations, failures, and limitations. All proposed quality targets are either met or explicitly marked as unmet.
- A working beta URL or clearly documented local authenticated demo is available, with reproducible setup instructions.
- Repository contains source, migrations, lockfiles, a safe sample corpus, gold dataset, evaluation runner, architecture, and deployment instructions.
- README explains the user problem and demonstrates the actual product; benchmark results are measured, not copied from the reference image.
- Three-minute recording and backup local demo are ready by 12 November; the last three days remain reserved for blockers.

Recommended demonstration

Show an upload and ready state, one exact-term question, one paraphrased question, and a citation opened in the source viewer. Ask an unsupported question to demonstrate refusal. Show a small development comparison and final benchmark counts, then state one known limitation. This demonstrates useful product behavior and engineering judgment in about three minutes.

14 Official technology references

Official documentation checked on 8 October 2026. These sources describe capabilities rather than guaranteeing this project's performance.

S1 Vite getting started
https://vite.dev/guide/

S2 FastAPI API documentation
https://fastapi.tiangolo.com/

S3 Supabase hybrid search
https://supabase.com/docs/guides/ai/hybrid-search

S4 pgvector project documentation
https://github.com/pgvector/pgvector

S5 Supabase RAG with permissions
https://supabase.com/docs/guides/ai/rag-with-permissions

S6 Docling formats and OCR configuration
https://github.com/docling-project/docling/blob/main/docs/usage/supported_formats.md

S7 Gemini model catalog
https://ai.google.dev/gemini-api/docs/models

S8 Gemini embeddings
https://ai.google.dev/gemini-api/docs/embeddings

S9 Ragas evaluation metrics
https://docs.ragas.io/en/latest/concepts/metrics/available_metrics/

S10 Supabase current pricing
https://supabase.com/pricing

S11 Gemini structured output
https://ai.google.dev/gemini-api/docs/structured-output

S12 pypdf text extraction
https://pypdf.readthedocs.io/en/stable/user/extract-text.html

Source capabilities support the proposed choices; timelines, acceptance thresholds, limits, budget, and cut rules are project recommendations. The attached project image supplies the original ingestion, retrieval, citation, and evaluation concept. Model and package availability must be rechecked when setup begins and before release.